

HelixTrack

HelixTrack

Tagline: JIRA alternative for the free world.

Summary: HelixTrack is a comprehensive, modern, open-source alternative to JIRA (and, via its Documents extension, Confluence) — a multi-platform project-management and issue-tracking system built on a Go microservices backend with native clients for web, desktop, and mobile.

Short description: Open-source JIRA/Confluence alternative. A Go microservices backend (“HelixTrack Core”) exposes a unified REST API for project and issue tracking, plus a Confluence-style document workspace, served to native Web, Desktop, Android, and iOS clients over HTTP/3 QUIC. (~40 words)

Long description: HelixTrack is an open-source project-management and issue-tracking platform positioned as a free-world alternative to JIRA and Confluence. Its heart is **HelixTrack Core**, a production-ready REST API microservice written in Go with the Gin framework, offering full issue tracking, agile/scrum boards, team management, and a hierarchical permissions engine with swappable local or HTTP-backed implementations. Core exposes a distinctive single, action-routed `/do` endpoint with a consistent request/response envelope, and integrates with decoupled Authentication, Permissions, and Localization services that communicate over HTTP/3 QUIC and can run on separate machines or clusters. Data is stored in SQLite for development and PostgreSQL in production, encrypted at rest with SQLCipher (AES-256). A **Documents V2** extension adds a Confluence-style workspace — spaces, pages, version control, templates, real-time WebSocket collaboration, and analytics. Around Core sit multiple client applications: an Angular web client, a Tauri + Angular desktop client, native Android (Kotlin) and iOS (Swift) apps, plus HarmonyOS and Aurora OS clients and a screensaver — all talking to the same backend and discoverable on local networks via UDP broadcast. The client applications are maintained as separate, private repositories and are presented here at the product level only. The project’s README cites extreme-

performance targets (50,000+ requests/second with sub-millisecond query times); these are stated design/marketing figures rather than independently published benchmarks.

Why we built it: To give teams a genuinely open, self-hostable replacement for the JIRA + Confluence stack — “for the free world” — without vendor lock-in, combining enterprise-grade tracking, documents, and collaboration under a permissive open-source license. (License note: CLAUDE.md states MIT, but the core/LICENSE file of record is Apache 2.0 — verify before publishing.)

Why it’s a game-changer: It collapses two heavyweight commercial products (issue tracking + wiki/documents) into one open, high-performance, self-hostable platform with true multi-platform native clients. The HTTP/3-everywhere, fully-decoupled microservices design and at-rest SQLCipher encryption target performance and security levels usually reserved for proprietary SaaS, while remaining open and free to run anywhere.

What’s innovative: - Unified action-based /do API — one endpoint, action-routed, with a consistent envelope, instead of sprawling REST surface area. - HTTP/3 QUIC as the default inter-service transport (documented ~30-50% latency reduction claim). - Swappable permissions engine (local or HTTP implementation) and optional, independently-deployable services (Auth, Permissions, Localization). - Multi-space data isolation via a --space-root flag (per-project isolated databases + assets). - SQLCipher AES-256 encryption at rest with a claimed <5% overhead. - Automatic client-to-server discovery via UDP broadcast on local networks. - Documents V2 “Confluence alternative” with optimistic-locking parallel editing and full change history.

Biggest technical challenges + how solved: - **Cross-platform native clients on one backend.** Solved by a strict single-backend contract (unified /do API + envelope) that every client — Web/Angular, Desktop/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS, Aurora — targets identically, with UDP-broadcast service discovery to locate Core. - **Low-latency, decoupled services.** Solved by adopting HTTP/3 QUIC across all inter-service calls and allowing each service (Auth, Permissions, Localization) to run independently and be disabled in test configs. - **Confluence-grade collaborative documents.** Documents V2 adds spaces/pages/versioning with optimistic locking, conflict detection, full change history, and real-time WebSocket sync. (Documented as ~95% complete with a known database field-mapping issue estimated at 8-10 hours to resolve — do not present as fully shipped.) - **Encryption without killing performance.** SQLCipher AES-256 at rest plus multi-layer caching (in-memory LRU + Redis in the Localization service) to offset overhead.

Tech stack (why + how): - **Go + Gin** — chosen for high-throughput, low-latency HTTP services; implements Core's REST API, middleware (JWT/CORS), and the /do router. - **HTTP/3 QUIC** — used as the transport between Core and its Auth/Permissions/Localization services to cut latency and enable connection resilience. - **PostgreSQL (prod) / SQLite (dev)** — relational store for the large tracking + documents schema; SQLite keeps local dev zero-setup, Postgres scales in production (via a production compose profile). - **SQLCipher (AES-256)** — transparent database-level encryption at rest for sensitive project data. - **Redis** — multi-layer caching in the Localization service (in-memory LRU + Redis) for fast multi-language lookups. - **Uber Zap + Lumberjack** — structured, rotated logging in Core. - **golang-jwt / JWT** — authentication and the jwt field in the API envelope. - **Angular 19 (+ Material, RxJS)** — the browser web client. - **Tauri 2.0 + Rust + Angular** — the cross-platform desktop client (small footprint, native shell). - **Kotlin (Android) / Swift + SwiftUI (iOS)** — native mobile clients. - **Docker / Docker Compose (Podman-compatible)** — containerized deployment (compose.helixtrack.yml, /health checks). - **Testify (Go); Cypress/Playwright/Karma+Jasmine (clients)** — layered automated testing across backend and clients.

Public links: - Core (backend), PUBLIC: github.com/Helix-Track/Core - Marketing/site: github.com/Helix-Track/Website and GitHub Pages target helix-track.github.io/Core - Community/contact: Telegram t.me/helixtrack; email svyaz.s.ulitkami@helixtrack.ru - **PRIVATE — do NOT deep-link:** the client applications (Web-Client, Desktop-Client, Android-Client, iOS-Client, Aurora-Client, Harmony-OS-Client) and the Everything umbrella/wrapper repo are private. Present clients at product level only ("native multi-platform clients"); optionally reference the Helix-Track GitHub org, not individual client repos.

Suggested diagrams/illustrations (OpenDesign): 1. **HelixTrack architecture map** — Core (Go/Gin) at center exposing the unified /do API, connected over HTTP/3 QUIC to decoupled Auth, Permissions, and Localization services, with PostgreSQL/SQLite (SQLCipher-encrypted) and Redis; native clients (Web, Desktop, Android, iOS, HarmonyOS, Aurora) fanning out via UDP discovery. 2. **Unified /do request/response envelope** — a clean before/after showing the single action-routed endpoint vs. a traditional sprawling REST surface (action/jwt/object/data → errorCode/errorMessage/data). 3. **JIRA + Confluence, unified** — a two-panel illustration: issue tracking / agile boards (Core) alongside the Documents V2 spaces/pages workspace, framed as one open platform. 4. **Multi-space isolation** — visual of --space-root producing per-project isolated databases + asset stores.

Site relevance: Both. Flagship product feature on **vasic.digital** (company); portfolio highlight on **milosvasic.ru** (personal CV).

Priority tier: Helix-primary. **Must be presentable and ranked BEFORE any ServerFactory projects.** This is the flagship of the Helix-Track product line.

Source provenance: Local repo /Volumes/T7/Projects/helix_track (the private Everything wrapper) — core/README.md, CLAUDE.md, AGENTS.md, core/LICENSE, compose.helixtrack.yml, .gitmodules, core/Website/README.md, WEBSITE_CONTENT_STRUCTURE.md. Cross-checked against harvested _analysis/github-helix-others.md and _analysis/MASTER-inventory.md. Caution flags (report softly or verify): performance figures (50k req/s, sub-ms) are claimed, not benchmark-backed; API-action / table / test counts differ across docs (e.g. 372 vs 387 API actions, 121 vs 125 tables); license MIT vs Apache-2.0 discrepancy (LICENSE file = Apache-2.0); Documents V2 ~95% complete. “Helix-Flow” appears in this repo only as an org name in governance files, not a HelixTrack component.